

Harshitha Mogili

2403A51L01

B-51

## ASSIGNMENT-16.2

### Database Design and Queries: Schema Design and SQL Generation

#### Task Descriptor-1: (Design Database Schema for Student Management System )

##### INPUT:

Schema SQL ●

```
1 CREATE TABLE Students (  
2     student_id INT PRIMARY KEY,  
3     name VARCHAR(100),  
4     email VARCHAR(100)  
5 );  
6  
7 CREATE TABLE Courses (  
8     course_id INT PRIMARY KEY,  
9     course_name VARCHAR(100),  
10    credits INT  
11 );  
12  
13 CREATE TABLE Enrollments (  
14     enrollment_id INT PRIMARY KEY,  
15     student_id INT,  
16     course_id INT,  
17     FOREIGN KEY (student_id) REFERENCES Students(student_id),  
18     FOREIGN KEY (course_id) REFERENCES Courses(course_id)  
19 );  
20  
21 -- SHOW OUTPUT  
22 SHOW TABLES;  
23 DESCRIBE Students;  
24 DESCRIBE Courses;  
25 DESCRIBE Enrollments;
```

##### OUTPUT:

| Results                                      |              |      |     |         |       |
|----------------------------------------------|--------------|------|-----|---------|-------|
| Query #5 <span>Execution time: 0.33ms</span> |              |      |     |         |       |
| Field                                        | Type         | Null | Key | Default | Extra |
| student_id                                   | int(11)      | NO   | PRI | null    |       |
| name                                         | varchar(100) | YES  |     | null    |       |
| email                                        | varchar(100) | YES  |     | null    |       |
| Query #6 <span>Execution time: 0.32ms</span> |              |      |     |         |       |
| Field                                        | Type         | Null | Key | Default | Extra |
| course_id                                    | int(11)      | NO   | PRI | null    |       |
| course_name                                  | varchar(100) | YES  |     | null    |       |
| credits                                      | int(11)      | YES  |     | null    |       |
| Query #7 <span>Execution time: 0.37ms</span> |              |      |     |         |       |
| Field                                        | Type         | Null | Key | Default | Extra |
| enrollment_id                                | int(11)      | NO   | PRI | null    |       |
| student_id                                   | int(11)      | YES  | MUL | null    |       |
| course_id                                    | int(11)      | YES  | MUL | null    |       |

## Explanation

1. Defines structure of database using tables: Students, Courses, Enrollments.
2. Primary keys ensure unique identification of each record.
3. Foreign keys maintain referential integrity between related tables.
4. Follows normalization (1NF, 2NF, 3NF) to remove redundancy and ensure consistency.

## Task Descriptor- 2(Insert Sample Records)

### INPUT:


Database: MySQL v5.7

Run
Update
Fork
Load Example
Star
PRO

Query SQL
PRO
Have any feedback?

```

1 INSERT INTO Students VALUES
2 (1, 'Arjun', 'arjun@gmail.com'),
3 (2, 'Meera', 'meera@gmail.com'),
4 (3, 'Rahul', 'rahul@gmail.com');
5
6 INSERT INTO Courses VALUES
7 (101, 'Database Systems', 4),
8 (102, 'Data Structures', 3),
9 (103, 'Operating Systems', 4),
10 (104, 'AI Fundamentals', 3);
11
12 INSERT INTO Enrollments VALUES
13 (1, 1, 101),
14 (2, 1, 102),
15 (3, 2, 103),
16 (4, 3, 101),
17 (5, 3, 104);
18
19 -- ☒ SHOW RESULT
20 SELECT * FROM Students;
21 SELECT * FROM Courses;
22 SELECT * FROM Enrollments;

```

### OUTPUT:

| Results    |       |                 | Copy as Markdown |
|------------|-------|-----------------|------------------|
| student_id | name  | email           |                  |
| 1          | Arjun | arjun@gmail.com |                  |
| 2          | Meera | meera@gmail.com |                  |
| 3          | Rahul | rahul@gmail.com |                  |

  

| Query #5 Execution time: 0.07ms |                   |         |  |
|---------------------------------|-------------------|---------|--|
| course_id                       | course_name       | credits |  |
| 101                             | Database Systems  | 4       |  |
| 102                             | Data Structures   | 3       |  |
| 103                             | Operating Systems | 4       |  |
| 104                             | AI Fundamentals   | 3       |  |

  

| Query #6 Execution time: 0.06ms |            |           |  |
|---------------------------------|------------|-----------|--|
| enrollment_id                   | student_id | course_id |  |
| 1                               | 1          | 101       |  |
| 2                               | 1          | 102       |  |
| 3                               | 2          | 103       |  |
| 4                               | 3          | 101       |  |
| 5                               | 3          | 104       |  |

## Explanation:

1. Populates tables with initial data using INSERT statements.
2. Ensures relational links by matching student\_id and course\_id.
3. Helps in testing database functionality with sample dataset.
4. SELECT queries verify whether data insertion is successful.

## Task Descriptor- 3: (CRUD Operations)

### INPUT:


Database: MySQL v5.7

Run
Update
Fork
Load Example
Star
PRO

Query SQL
PRO
Have any feedback?

```

1 -- READ
2 SELECT * FROM Courses;
3
4 -- UPDATE
5 UPDATE Students
6 SET email = 'arjun_new@gmail.com'
7 WHERE student_id = 1;
8
9 -- DELETE
10 DELETE FROM Enrollments
11 WHERE enrollment_id = 5;
12
13 -- SHOW RESULT
14 SELECT * FROM Students; -- shows updated email
15 SELECT * FROM Enrollments; -- shows deleted record

```

## OUTPUT:

|                                       |                        |           |                  |
|---------------------------------------|------------------------|-----------|------------------|
| Results                               |                        |           | Copy as Markdown |
| Query #1                              | Execution time: 0.13ms |           |                  |
| course_id                             | course_name            | credits   |                  |
| There are no results to be displayed. |                        |           |                  |
| Query #2                              | Execution time: 0.13ms |           |                  |
| There are no results to be displayed. |                        |           |                  |
| Query #3                              | Execution time: 0.07ms |           |                  |
| There are no results to be displayed. |                        |           |                  |
| Query #4                              | Execution time: 0.06ms |           |                  |
| student_id                            | name                   | email     |                  |
| There are no results to be displayed. |                        |           |                  |
| Query #5                              | Execution time: 0.05ms |           |                  |
| enrollment_id                         | student_id             | course_id |                  |
| There are no results to be displayed. |                        |           |                  |

## Explanation:

1. SELECT retrieves data from tables for viewing records.
2. UPDATE modifies existing data like student email.
3. DELETE removes specific records such as enrollments.
4. Validates operations using SELECT to ensure correctness.

## Task Descriptor- 4: (Aggregate Queries)

## INPUT:

Database: MySQL v5.7 Run Update Fork Load Example Star PRO

Query SQL PRO

Query successfully executed in 50.44ms

```
1 -- Count students per course
2 SELECT c.course_name, COUNT(e.student_id) AS student_count
3 FROM Courses c
4 JOIN Enrollments e ON c.course_id = e.course_id
5 GROUP BY c.course_name;
6
7 -- Courses with more than 1 student
8 SELECT c.course_name
9 FROM Courses c
10 JOIN Enrollments e ON c.course_id = e.course_id
11 GROUP BY c.course_name
12 HAVING COUNT(e.student_id) > 1;
```

## OUTPUT:

Database: MySQL v5.7 Run Update Fork Load Example Star PRO

Results PRO Copy as Markdown

Query #1 Execution time: 0.2ms

| course_name                           | student_count |
|---------------------------------------|---------------|
| There are no results to be displayed. |               |

Query #2 Execution time: 0.16ms

| course_name                           |
|---------------------------------------|
| There are no results to be displayed. |

### Explanation:

1. Uses functions like COUNT() to summarize data.
2. GROUP BY groups records based on course names.
3. HAVING filters grouped data (e.g., courses with >1 student).
4. Helps in analysis like enrollment trends per course.

### Task Descriptor -5: (Query Optimization)

#### INPUT:

Database: MySQL v5.7 Run Update Fork Load Example Star PRO

Query SQL PRO Query successfully executed in 8.19ms

```

1 -- Original
2 SELECT * FROM Students
3 WHERE student_id IN (
4     SELECT student_id FROM Enrollments WHERE course_id = 101
5 );
6
7 -- Optimized
8 SELECT s.*
9 FROM Students s
10 JOIN Enrollments e ON s.student_id = e.student_id
11 WHERE e.course_id = 101;
  
```

## OUTPUT:

The screenshot shows a web-based SQL execution environment. At the top, there's a blue header bar with a database icon, the text 'Database: MySQL v5.7', and several action buttons: 'Run', 'Update', 'Fork', 'Load Example', 'Star', and a 'PRO' badge. Below the header, the 'Results' section is visible, also featuring a 'PRO' badge and a 'Copy as Markdown' button. Two queries are listed. Query #1 has an execution time of 0.25ms and shows a table with columns 'student\_id', 'name', and 'email'. Below the table, it states 'There are no results to be displayed.' Query #2 has an execution time of 0.14ms and shows an identical table structure, also stating 'There are no results to be displayed.'

Database: MySQL v5.7 ▼ Run Update Fork Load Example Star PRO

Results PRO Copy as Markdown ↗

Query #1 Execution time: 0.25ms

| student_id | name | email |
|------------|------|-------|
|------------|------|-------|

There are no results to be displayed.

Query #2 Execution time: 0.14ms

| student_id | name | email |
|------------|------|-------|
|------------|------|-------|

There are no results to be displayed.

## Explanation:

1. Identifies inefficient queries (e.g., subqueries).
2. Rewrites queries using JOINS for better performance.
3. Ensures both original and optimized queries give same result.
4. Improves execution speed, especially for large datasets.